

# Manejo de archivos



Un archivo es una **colección de datos almacenados** en un sistema de almacenamiento (DH, SSD, USB, etc). Permiten a los usuario y a los programas **guardar y recuperar** información de manera organizada y persistente. Los archivos pueden contener diversos tipos de datos, como texto, imágenes, audio, video, código fuente, etc.



Los **archivos de texto** contienen caracteres legibles, es posible no solo leer dicho contenido sino también modificarlo usando un editor de texto.



Un **archivo binario** es cualquier archivo que no se pueda interpretar en forma de texto: una imagen, un sonido o incluso un archivo comprimido.



1. **Apertura:** Acceder a un archivo para leer, escribir o modificar su contenido.
2. **Lectura:** Extrae datos de un archivo.
3. **Escritura:** Guardar datos en un archivo, ya sea creando contenido nuevo o modificando el existente.
4. **Cierre:** terminar el acceso a un archivo y liberar cualquier recurso asociado.



# Modo de apertura



utilizar el modo  
correspondiente para  
cada caso

| MODO | DESCRIPCIÓN                                                                  |
|------|------------------------------------------------------------------------------|
| r    | Abre un archivo de texto sólo para lectura                                   |
| rb   | Abre un archivo binario sólo para lectura                                    |
| r+   | Abre un archivo de texto para escritura y lectura                            |
| w    | Abre un archivo de texto sólo para escritura (si existe lo sobrescribe)      |
| wb   | Abre un archivo sólo para escritura binaria (si existe lo sobrescribe)       |
| w+   | Abre un archivo de texto para escritura y lectura (si existe lo sobrescribe) |
| a    | Abre un archivo para anexar información                                      |

**r** -> abre un archivo sólo para lectura. El puntero al archivo está localizado al comienzo del archivo. Este es el modo predeterminado de la función.

**r+** -> abre un archivo para escritura y lectura. El puntero del archivo está localizado al comienzo del archivo.



**w** -> abre un archivo solo para escritura. Sobreescribe el archivo si este ya existe. Si el archivo no existe lo crea.

**w+** -> abre un archivo para escritura y lectura. Sobreescribe el archivo si este ya existe. Si el archivo no existe lo crea.

**a** -> abre un archivo para anexo. El puntero del archivo está al final del archivo si este existe. Es decir, el archivo está en modo anexo. Si el archivo no existe, crea un nuevo archivo para escritura.

# Para abrir un archivo con Python, podemos usar la función **open**.

```
archivo = open(nombre_archivo, modo)
```

**archivo** objeto file, el cual será utilizado para llamar a otros métodos asociados con el archivo.

**nombre\_archivo** string que contiene el nombre del archivo al que queremos acceder.

**modo** string que contiene el modo de apertura del archivo.

# Modo de cierre



Para cerrar un archivo con Python, podemos usar la función **close**.

```
archivo.close()
```

**archivo** objeto file que fue obtenido con el método open.

Podemos usar la sentencia **with** para abrir archivos en Python y dejar que el intérprete se encargue de cerrar el mismo.

```
with open('archivo.txt', 'r+') as archivo:  
    for linea in archivo:  
        print(linea, end="")
```

# Modo de escritura



**Write** : Escribe una cadena de caracteres dentro del archivo.

```
archivo = open('archivo.txt', 'w')
archivo.write('Primer linea de texto\n')
archivo.write('segunda linea\n')
archivo.write('tercera linea\n')
archivo.close()
```



**Writelines** : Escribe una lista de cadena de caracteres dentro del archivo. El parámetro que recibe el método podrá ser cualquier iterable.

```
archivo = open('archivo.txt', 'w')
lineas_texto = ['Primer linea de texto\n',
                'segunda linea\n',
                'tercera linea\n']
archivo.writelines(lineas_texto)
archivo.close()
```

# Modo de lectura



**read:** permite extraer un string que contenga todos los caracteres del archivo(Es posible limitar al método read a que lea cierta cantidad de bytes **read(cantidad)**).

```
# Abrimos el archivo en modo lectura y escritura
archivo = open('archivo.txt', 'r+')
texto = archivo.read()
print('El contenido del archivo es: ' + texto)
# Cerramos el archivo
archivo.close()
```

**readlines:** Permite obtener una lista con todas las líneas que contiene el archivo.

```
archivo = open('archivo.txt', 'r+')
lista_lineas = archivo.readlines()
for linea in lista_lineas:
    print(linea, end="")
# Cerramos el archivo
archivo.close()
```

# Archivos CSV



**CSV:** (Comma Separated Values) es un formato de archivo de texto que almacena datos en tablas, donde cada línea representa una fila y los valores dentro de cada fila están separados por comas.

```
Nombre,Edad,Ciudad  
Juan,28,Madrid  
Ana,22,Barcelona  
Luis,35,Sevilla
```

## Lectura:

Podemos leer un csv recorriendo sus líneas y obteniendo uno por uno los valores que se encuentran entre las comas.

```
leer csv.py > ...
1  with open("archivo.csv", "r") as archivo:
2      matriz = []
3      nombres_columnas = archivo.readline().strip().split(",")
4
5      for linea in archivo:
6
7          linea = linea.rstrip("\n")
8          fila = []
9          valores = linea.split(",")
10
11         for valor in valores:
12
13             if valor.isdigit():
14                 fila.append(int(valor))
15             else:
16                 fila.append(valor)
17
18         matriz.append(fila)
19
20     print(nombres_columnas)
21
22     for fila in matriz:
23         print(fila)
```

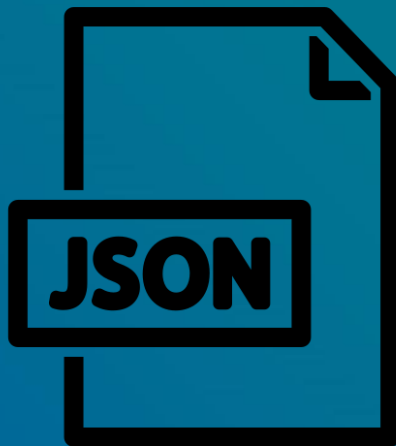
## Escritura:

Para guardar un csv debemos realizar el procedimiento inverso. Debemos transformar cada fila de la matriz en una línea de texto en la que los valores se separan por comas. Y cuando la fila finaliza, realizar un salto de línea.

```
1 nombres_columnas = ["Nombre", "Edad", "Ciudad"]
2 matriz = [["Pedro", 24, "París"], ["José", 25, "toronto"]]
3
4 with open("archivo.csv", "w") as archivo:
5     archivo.write(",".join(nombres_columnas) + "\n")
6
7     for fila in matriz:
8         linea = ""
9         for i in range(len(fila)):
10
11             linea += str(fila[i])
12
13             if i < (len(fila) - 1):
14                 linea += ","
15
16         archivo.write(linea + "\n")
```



# Archivos JSON



**Json:** (JavaScript Object Notation) es un formato de texto utilizado para almacenar e intercambiar datos estructurados. Organiza los datos en pares de clave-valor, similar a un diccionario en Python.

```
{  
  "nombre": "Juan",  
  "edad": 28,  
  "ciudad": "Madrid",  
  "amigos": [  
    {"nombre": "Ana", "edad": 22},  
    {"nombre": "Luis", "edad": 35}  
  ]  
}
```

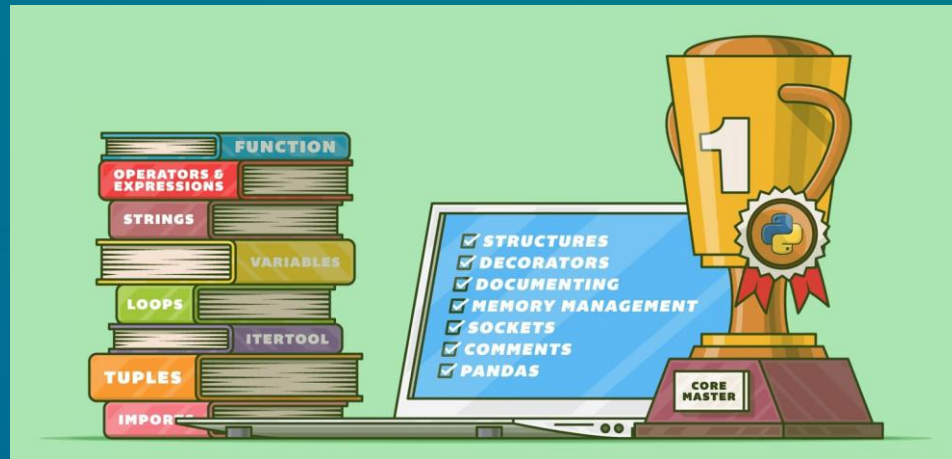
**Lectura:** El módulo json en Python permite trabajar con datos en formato JSON. La función json.load() lee un archivo JSON y lo convierte en un objeto de Python (como un diccionario o una lista).

```
import json
# Leer archivo JSON
with open('archivo.json', 'r') as archivo_json:
    datos = json.load(archivo_json) # Cargar el contenido en un diccionario
    print(datos)
```

**Escritura:** La función `json.dump()` convierte un objeto de Python (como un diccionario) a formato JSON y lo escribe en un archivo.

```
import json
# Escribir en un archivo JSON
datos = {
    "nombre": "Juan",
    "edad": 28,
    "ciudad": "Madrid"
}
with open('archivo.json', 'w') as archivo_json:
    json.dump(datos, archivo_json, indent=4) # El parámetro indent para que sea más legible
```

# Paradigma Funcional

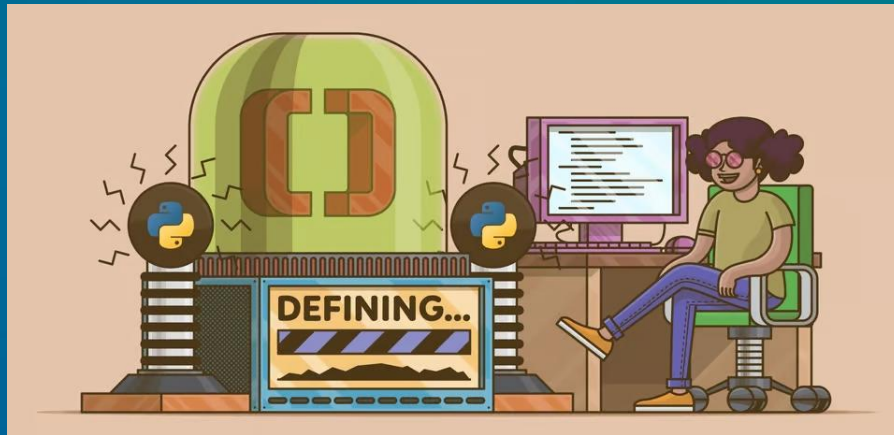


- El concepto **programación funcional** hace referencia a un estilo de programación que utiliza las funciones como unidad básica de código.
- Hay desde lenguajes puramente funcionales como Haskell o Lisp, hasta lenguajes multiparadigma como **Python**.
- Para que un lenguaje permita la programación funcional, **debe tratar a las funciones como ciudadanos de primera clase**. Es lo que ocurre con Python; las funciones son objetos, al igual que los strings, los números y las listas. Las funciones pueden pasarse como parámetros a otras funciones o devolverse como valores de retorno de funciones.

## Características principales:

- Funciones puras: Una función pura es aquella cuyo resultado depende solo de sus parámetros de entrada y no tiene efectos secundarios.
- Hacer código más declarativo: Se enfoca en el "qué" en lugar del "cómo", lo que suele resultar en un código más claro y fácil de entender.
- Funciones de orden superior: Estas son funciones que pueden recibir otras funciones como argumentos o devolver funciones como resultado.
- Expresiones en lugar de instrucciones: En lugar de usar bucles y comandos, se favorecen las expresiones y combinaciones de funciones.

# Funciones como Ciudadanos de Primera Clase





Se dice que un lenguaje de programación tiene funciones de primera clase si se admite:

- Asignar funciones a variables o almacenarlas en estructuras de datos.
- Poder pasar funciones como argumentos a otras funciones.
- Que el valor de retorno de una función sea otra función.

Las funciones en Python son ciudadanos de primera clase (first-class citizens). La máxima de Python de “**Everything is an object**” (todo es un objeto) también vale para las funciones.

- Asignar funciones a variables



```
def sumar(a, b):  
    return a + b  
  
operacion = sumar  
  
print(operacion(4, 2)) # 6
```

- Almacenarlas en estructuras de datos (listas)

```
def sumar(a, b):  
    return a + b  
  
def restar(a, b):  
    return a - b  
  
def multiplicar(a, b):  
    return a * b  
  
operaciones = [sumar, restar, multiplicar]  
  
for operacion in operaciones:  
    print(operacion(10, 5))
```

- Almacenarlas en estructuras de datos (tuplas)

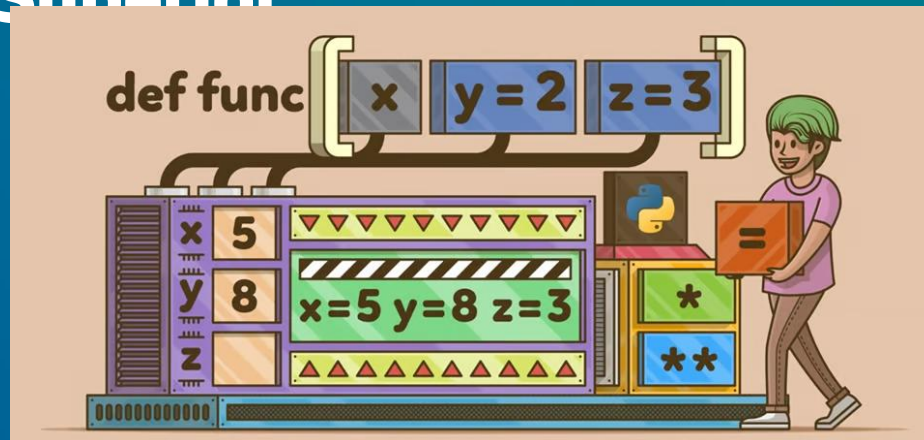
```
def sumar(a, b):  
    return a + b  
  
def restar(a, b):  
    return a - b  
  
def multiplicar(a, b):  
    return a * b  
  
operaciones = (sumar, restar, multiplicar)  
  
resultado_suma = operaciones[0](10, 5)  
resultado_resta = operaciones[1](10, 5)  
  
print(f"Resultado de la suma: {resultado_suma}")  
print(f"Resultado de la resta: {resultado_resta}")
```

- Almacenarlas en estructuras de datos (diccionarios)

```
def sumar(a, b):  
    return a + b  
  
def restar(a, b):  
    return a - b  
  
def multiplicar(a, b):  
    return a * b  
  
operaciones = {  
    "sumar": sumar,  
    "restar": restar,  
    "multiplicar": multiplicar  
}  
  
resultado = operaciones["multiplicar"](10, 5)  
print(f"Resultado de la multiplicación: {resultado}")
```

Esto demuestra la flexibilidad de las funciones como ciudadanos de primera clase en Python, permitiendo manipularlas y almacenarlas como cualquier otro tipo de dato.

# Funciones de Orden Superior



Las funciones de orden superior (o funciones de mayor orden) son aquellas que pueden tomar otras funciones como argumentos y/o devolver funciones como resultados. Son una característica poderosa que permite una gran flexibilidad y funcionalidad en el código.



# Funciones como parámetro de otras funciones

En Python, las funciones pueden ser utilizadas como parámetros de otras funciones. Esto es posible porque las funciones en Python, como mencionamos anteriormente, son ciudadanos de primera clase.

- Función que toma otra función como parámetro

```
def sumar(x, y):  
    return x + y  
  
def restar(x, y):  
    return x - y  
  
def operar(a, b, funcion):  
    return funcion(a, b)  
  
resultado_suma = operar(10, 5, sumar)  
resultado_resta = operar(10, 5, restar)  
  
print(f"Resultado de la suma: {resultado_suma}") # Resultado de la suma: 15  
print(f"Resultado de la resta: {resultado_resta}") # Resultado de la resta: 5
```

```
def aplicar_operacion(numero, funcion):  
    resultado = funcion(numero)  
    return resultado  
  
def duplicar(x):  
    return x * 2  
  
def triplicar(x):  
    return x * 3  
  
# Usando la función aplicar_operacion  
resultado1 = aplicar_operacion(5, duplicar)  
resultado2 = aplicar_operacion(5, triplicar)  
  
print(resultado1) # Imprime 10  
print(resultado2) # Imprime 15
```

- Función que toma otra función como parámetro

```
def ordenar_lista(funcion, lista):  
    """Ordena una lista según una función de comparación."""  
    lista.sort(key=funcion)  
    return lista  
  
def ordenar_por_longitud(cadena):  
    return len(cadena)  
  
palabras = ["manzana", "banana", "pera"]  
resultado = ordenar_lista(ordenar_por_longitud, palabras)  
print(resultado)  
# Salida: ['pera', 'banana', 'manzana']
```

- Función que toma otra función como parámetro

```
def filtrar_lista(funcion, lista):  
    """Filtra una lista según una función de predicado."""  
    resultado = []  
    for elemento in lista:  
        if funcion(elemento):  
            resultado.append(elemento)  
    return resultado  
  
def es_par(numero):  
    return numero % 2 == 0  
  
numeros = [1, 2, 3, 4, 5]  
resultado = filtrar_lista(es_par, numeros)  
print(resultado)  
# Salida: [2, 4]
```

# Funciones como retorno de otras funciones

Es posible también, que una función retorne otra función como resultado. Este tipo de funciones se llaman funciones que retornan funciones.

Este tipo de funciones son muy útiles para crear decoradores o para generar funciones personalizadas.

- Función que devuelve otra función

```
def funcion_externa(x):  
    return x + 5  
  
def retornar_funcion():  
    return funcion_externa  
  
# Obtener la función externa  
mi_funcion = retornar_funcion()  
  
# Usar la función externa  
resultado = mi_funcion(10)  
print(resultado) # Salida: 15
```

```
def crear_multiplicador_con_base(base):  
    def multiplicar_con_base(x):  
        return base * x  
    return multiplicar_con_base  
  
# Crear una función que multiplica por 10  
multiplicador_por_10 =  
    crear_multiplicador_con_base(10)  
  
# Usar la función generada  
resultado = multiplicador_por_10(7)  
print(resultado) # Salida: 70
```



- Función que devuelve otra función que multiplica su entrada por una base dada.

```
def crear_multiplicador_con_base(base):  
    def multiplicar_con_base(x):  
        return base * x  
    return multiplicar_con_base  
  
# Crear una función que multiplica por 10  
multiplicador_por_10 =  
    crear_multiplicador_con_base(10)  
  
# Usar la función generada  
resultado = multiplicador_por_10(7)  
print(resultado) # Salida: 70
```

- Función que retorna diferentes funciones de formateo de cadenas

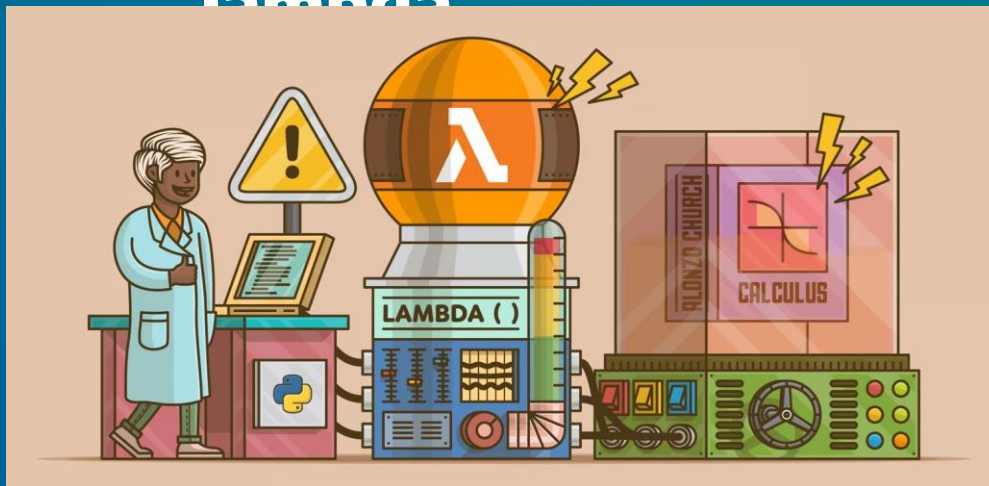
1

```
def formatear_mayusculas(texto):  
    return texto.upper()  
  
def formatear_minusculas(texto):  
    return texto.lower()  
  
def formatear_titulo(texto):  
    return texto.title()  
  
# Función que retorna la función de formateo deseada  
def obtener_formateador(tipo):  
    if tipo == 'mayusculas':  
        return formatear_mayusculas  
    elif tipo == 'minusculas':  
        return formatear_minusculas  
    elif tipo == 'titulo':  
        return formatear_titulo  
    else:  
        raise ValueError("Tipo de formateo no válido")
```

2

```
# Ejemplo de uso  
texto = "hola mundo"  
  
# Obtener la función de formateo deseada  
formateador = obtener_formateador('mayusculas')  
resultado = formateador(texto)  
print(f"Texto en mayúsculas: {resultado}") # Salida: HOLA MUNDO  
  
formateador = obtener_formateador('minusculas')  
resultado = formateador(texto)  
print(f"Texto en minúsculas: {resultado}") # Salida: hola mundo  
  
formateador = obtener_formateador('titulo')  
resultado = formateador(texto)  
print(f"Texto en título: {resultado}") # Salida: Hola Mundo
```

# Funciones Lambda



Las funciones lambda o anónimas son un tipo de funciones en Python que típicamente se definen en una línea y cuyo código a ejecutar suele ser pequeño.

## Características

- Funciones anónimas: Las funciones lambda son funciones anónimas, es decir, no tienen un nombre definido.
- Sintaxis concisa: Se definen en una sola línea utilizando la palabra clave lambda seguida de los parámetros, un signo : y una expresión.
- Retorno implícito: devuelven automáticamente el resultado de la expresión, sin necesidad de utilizar la palabra clave return.
- Limitación en la lógica: Están diseñadas para operaciones simples y no soportan múltiples líneas o declaraciones complejas.

## Características

- No recomendadas para lógica compleja: Debido a su sintaxis limitada, no se recomienda usarlas para lógica compleja o extensa.
- Útiles en expresiones cortas: Son ideales para ser usadas en casos donde se requiere una función pequeña y de uso inmediato.
- Usos comunes: Se utilizan frecuentemente en funciones de orden superior como `map()`, `filter()`, y `sorted()`, donde se requieren funciones cortas.

- Sintaxis de una función lambda



```
suma = lambda a, b: a + b  
print(suma(2, 4)) # 6
```



```
suma = (lambda a, b, c: a + b + c)(2, 3, 4)  
print(suma) # 9
```

Si una función lambda no se guarda en una variable o no se pasa directamente como argumento a otra función, se crea pero no se puede utilizar porque no está referenciada en ninguna parte.

- Comparando una función normal con una función lambda

```
● ● ●  
  
# funcion normal  
def suma(a, b):  
    return a + b  
  
resultado = suma(3, 5)  
print(resultado) # Salida: 8  
  
# funcion lambda  
suma_lambda = lambda a, b: a + b  
  
resultado_lambda = suma_lambda(3, 5)  
print(resultado_lambda) # Salida: 8
```

- Lo que sería una función que suma dos números como la que vemos primero en el código.
- Se podría expresar en forma de una función lambda.



- lambda como parámetro de una función



```
def validar(valor, condicion):  
    return condicion(valor)  
  
es_positivo = lambda x: x > 0  
print(validar(10, es_positivo)) # Salida: True  
print(validar(-5, es_positivo)) # Salida: False
```

En este ejemplo la función validar acepta dos parámetros, uno es un entero y el otro una función. En este caso le pasamos una función lambda como segundo parámetro.

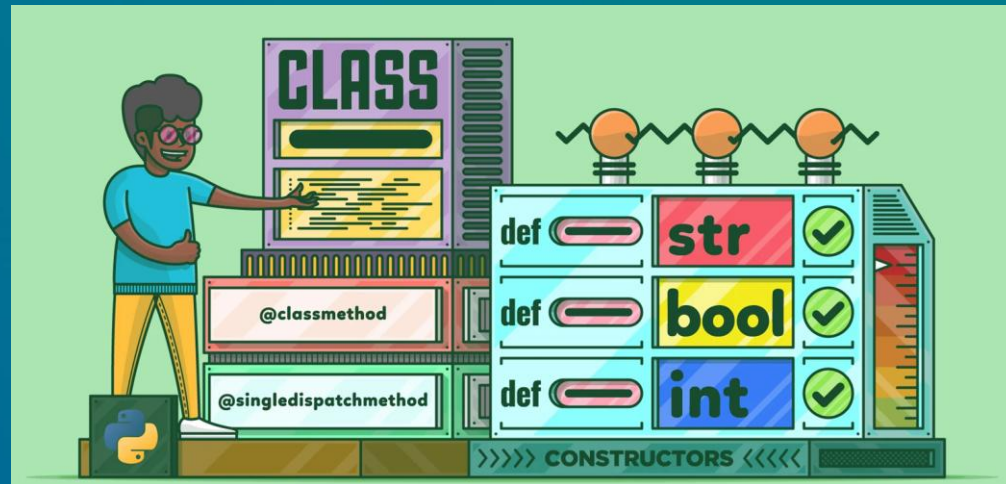
# Resumen de la clase:

- Funciones como ciudadanos de primera clase: Las funciones en Python pueden ser asignadas a variables, pasadas como argumentos y retornadas como resultado.
- Funciones como parámetros: Permiten crear soluciones más flexibles y reutilizables al pasar funciones como argumentos.
- Funciones con retorno de otras funciones: Permiten generar funciones especializadas en tiempo de ejecución.
- Funciones lambda: Son expresiones concisas para crear funciones anónimas, útiles en situaciones donde se requieren funciones simples y cortas.

## Recomendaciones:

- **Practicar** el uso de funciones como ciudadanos de primera clase.
- **Pasar** funciones como parámetros para mayor flexibilidad.
- **Explorar** el retorno de funciones para crear lógica modular.
- **Dominar** las funciones lambda para mantener el código limpio.

# Programación orientada a objetos



# Paradigma orientado a objetos

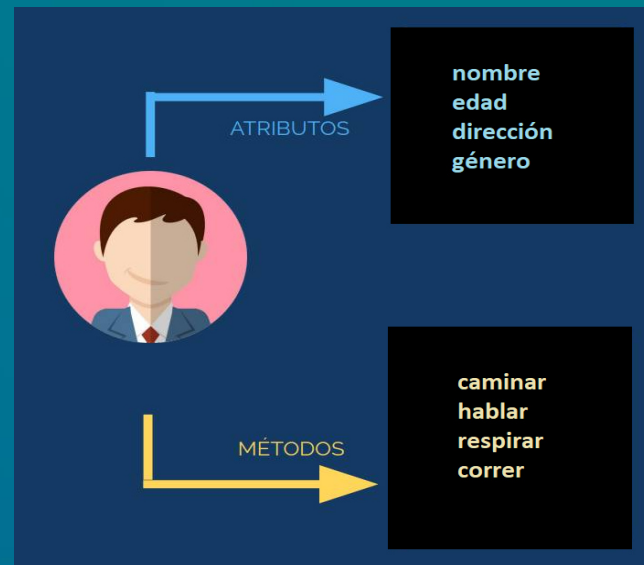
Este paradigma responde a una forma de encarar el desarrollo de software. El mismo se basa en la definición de objetos que forman parte de un dominio en cuestión, interactuando con otros objetos que se comunican por medio de mensajes. Por lo tanto, la programación orientada a objetos define la forma de producir código bajo este paradigma.

# Pilares de P00

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| Abstracción     | Proceso de definir los atributos y métodos de una clase.                    |
| Encapsulamiento | Protege la información de manipulaciones no autorizadas                     |
| Polimorfismo    | Da la misma orden a varios objetos para que respondan de diferentes maneras |
| Herencia        | Las clases hijas heredan atributos y métodos de las clases padre.           |

# ¿Qué son los objetos?

Un objeto es la representación de una abstracción (una clase) en memoria. En un programa podría ser una persona con **características (atributos)** como nombre, edad y dirección, y **comportamientos (métodos)** como caminar, hablar, respirar y correr.



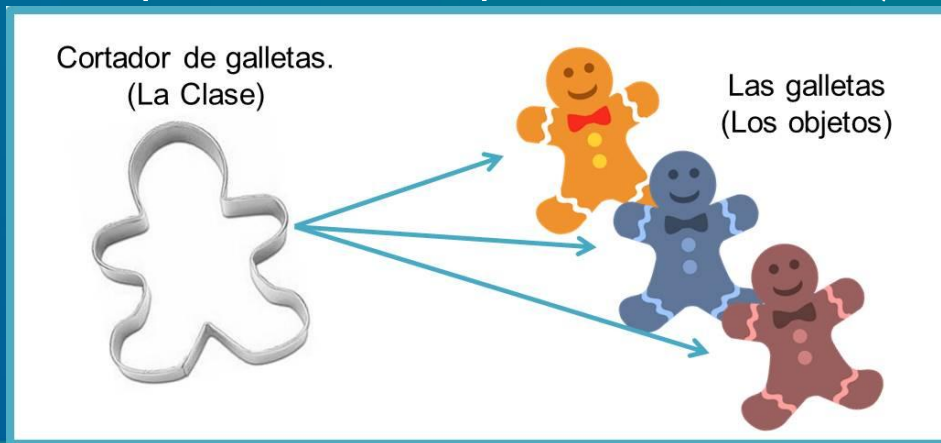
# Comunicación entre objetos

Los objetos no trabajan de forma aislada, sino que se comunican entre sí mediante el envío de mensajes. Estos se dan cuando un objeto invoca un método de otro para solicitarle una acción o información. Este intercambio permite que los objetos trabajen juntos para resolver problemas más complejos, manteniendo independencia y encapsulamiento.



# ¿Qué es una clase?

Podemos definir a una clase como un molde que **definirá** las características y el comportamiento que **tendrá** un objeto



# Cómo definir una clase

Todas las definiciones de clase comienzan con la palabra clave **class**, seguida del nombre de la clase y dos puntos.

```
class Personaje:  
    pass
```

Los nombres de las clases de python se escriben en **CapitalizedWords** (UpperCamelCase)

# ¿Cómo definir atributos?

A las características de un objeto las vamos a llamar atributos. Estos atributos los tendrán todos los objetos de tipo Personaje. Los mismos se definirán en un método llamado `__init__` (Constructor de la clase)

```
class Personaje:
    def __init__(self, id, nombre, nano, vuela) -> None:
        self.id = id # crea un atributo id y le asigna el valor id.
        self.nombre = nombre
        self.usa_nanotecnologia = nano
        self.puede_volar = vuela
```

# Crear una instancia

La creación de un nuevo objeto a partir de una clase se denomina instanciación de un objeto: se crea el objeto en memoria (por eso se dice que un objeto es una instancia de una clase)

Se puede instanciar un nuevo **Personaje** escribiendo el nombre de la clase, seguido de paréntesis de apertura y cierre:

```
personaje_A = Personaje(1, 'IronMan', 'No', 'Si')  
personaje_B = Personaje(2, 'IronSpider', 'Si', 'No')
```

# Acceder a los atributos

Se puede acceder a sus atributos de instancia mediante la notación de puntos:

```
personaje_A = Personaje(1, 'IronMan', 'No', 'Si')
personaje_B = Personaje(2, 'IronSpider', 'Si', 'No')

print(personaje_A.nombre) # IronMan
print(personaje_B.nombre) # IronSpider
```

# Cómo definir métodos

Los métodos de instancia representan las acciones que pueden realizar los objetos. Son funciones que se definen dentro de una clase y solo se pueden llamar desde una instancia de esa clase. Al igual que `__init__()`, el primer parámetro de un método de instancia siempre es `self`.

```
class Personaje:
    def __init__(self, id, nombre, nano, vuela) -> None:
        self.id = id # crea un atributo id y le asigna el valor id.
        self.nombre = nombre
        self.usa_nanotecnologia = nano
        self.puede_volar = vuela

    def retornar_descripcion(self) -> str:
        return '{0}-{1}'.format(self.id, self.nombre)
```

# Llamar a un método

En la clase Personaje, el método `retornar_descripcion` devuelve una cadena que contiene el id y el nombre del personaje.

```
personaje_A = Personaje(1, 'IronMan', 'No', 'Si')
personaje_B = Personaje(2, 'IronSpider', 'Si', 'No')

print(personaje_A.retornar_descripcion())# 1-IronMan
print(personaje_B.retornar_descripcion())# 2-IronSpider
```

# Strings





# ¿Qué son los strings?

“String” es el nombre más común que se da a las **cadenas de caracteres**. Las strings son tipos de **datos simples** que a menudo se abrevian como **str**. En Python, existen varios **métodos** específicas para manipular strings.

### Contenido

- **Formato** -> upper, lower, capitalize, title, strip.
- **Modificación** -> replace, split, join, zfill.
- **Búsqueda** -> isalpha, isalnum, count, find.

## Formato

- **Upper** pone todas las letras de la string en **mayúsculas**.
- **Lower** pone todas las letras de la string en **minúsculas**.
- **Capitalize** hace el **primer carácter** de la string **mayúscula**.
- **Title** hace que las **primeras letras** de **todas las palabras** sean **mayúsculas**.
- **Strip** saca los espacios vacíos que haya **antes y después** de cualquier carácter.

### Formato

```
cadena = "a formatear!"  
cadena_strip = "    espacios    "  
print(f"Upper: {cadena.upper()}")  
print(f"Lower: {cadena.lower()}")  
print(f"Capitalize: {cadena.capitalize()}")  
print(f"Title: {cadena.title()}")  
print(f"Strip: {cadena_strip.strip()}")
```



```
Upper: A FORMATEAR!  
Lower: a formatear!  
Capitalize: A formatear!  
Title: A Formatear!  
Strip: espacios
```

## Modificación - Replace

Reemplaza la cadena original con la cadena deseada en esa instancia.

```
cadena = "Python es horrible"  
print(cadena.replace("horrible", "genial"))
```



Python es genial

## Modificación - Split

Toma un **separador** y **corta** la cadena donde sea que encuentre ese separador.

```
cadena = "Do-Re-Mi-Fa-Sol-La-Si"  
print(cadena.split('-'))
```



```
['Do', 'Re', 'Mi', 'Fa', 'Sol', 'La', 'Si']
```

## Modificación - Join

Une las cadenas especificadas poniendo el **separador** deseado de por medio

```
separador = " de "  
cadena_uno = "11"  
cadena_dos = "Enero"  
cadena_tres = "1997"  
resultado = separador.join([cadena_uno, cadena_dos, cadena_tres])  
print(resultado)
```



11 de Enero de 1997

## Modificación - zfill

Llena de ceros el lado izquierdo de la cadena hasta tener la misma el largo especificado

```
cadena = "4971"  
cadena = cadena.zfill(6)  
print(cadena)
```



004971



## Búsqueda - isalpha/isalnum

La función **isalpha** y la función **isalnum** toman una string y devuelven un **booleano**. **isalpha** devuelve True si **todos** los caracteres de la string son **letras** e **isalnum** devuelve True si **todos** los caracteres son **alfanuméricos**.

```
cadena = "Holamundo222"  
print(cadena.isalpha()) #False  
print(cadena.isalnum()) #True
```

## Búsqueda - Count

**Count** nos permite **contar** la cantidad de veces que una cierta combinación de caracteres (O un carácter solo) **aparece** en una string.

```
cadena = "karma karma karma chameleon"  
print(cadena.count("karma")) #Print: 3
```

## Búsqueda - Find

**Find** devuelve en qué carácter está la **combinación de caracteres** buscada. También puede buscar una combinación **a partir de cierto carácter** de la string, o buscar de **adelante hacia atrás**.